

Appendix B: The Spec Session, a working template

A starting shape, frozen at printing. Take it, run it, and rebuild it for your team.

I don't have years of experience running these. Nobody does, because the ritual is younger than the problem it solves. What follows is a construction from the rituals we know, and it inherits from refinement, not sprint planning. Capacity stops being the constraint when the agent implements, so the capacity arithmetic of planning has nothing left to do. What survives is refinement's content with mob programming's mechanics. Run it, break it, tell me where it fails.

Cadence and size

60 to 90 minutes, hard stop. Overrun means the ticket wasn't ready or is too big, and both are findings, not failures.

This is not a weekly ceremony. A team might run two or three of these in a day, with real breaks in between, because the session is the work now, not the meeting before the work. Mob programming already taught this lesson: sustained group thinking exhausts faster than solo implementation, and a tired room produces a worse spec. Some days most of the day is spec sessions, and that's not planning overhead eating the team's time, that's the team doing engineering.

Three to seven people. Below three, someone is prompting with witnesses. That rule is about the standing session, not the first try: the pair experiment the Agora's door opens with is deliberately below this floor, and it's how you find out whether your team wants the real thing. Above seven, the session splits by domain, and the team accepts the coverage tension that comes with splitting.

One session, one spec. No batch grooming of a backlog.

Roles, rotating per session

The session lead owns the room that day, not the product forever. They close discussions, call the timebox, and decide when a disagreement gets logged instead of resolved.

The driver types the spec on the shared screen and types only what the group says. The driver's confusion is a feature, because a sentence the driver can't write down is a sentence the agent can't run with. This is strong-style pairing's law applied to planning, for an idea to reach the document it must pass through someone else's hands, and the credit belongs to the mob lineage of Woody Zuill and Llewellyn Falco.

If you use AI during specing, you use it together. One model, on the shared screen, driven by the driver. It can challenge the spec, surface prior art, play the agent reading the draft. What it can't be is a private second opinion, because the moment people validate against their own tab instead of the room, the session is just parallel prompting with attendance. This AI is not the agent that runs after the gate, it's a tool the group wields while the spec is still soft.

This rule exists because of what a single private prompt did to a mob session. The Alexandria Problem carries the scene.

Everyone else navigates. Intent, edge cases, system constraints.

The Stakeholder Navigator is deliberately absent unless intent needs clarifying. Their job is protecting the room, and their presence inside it turns the session back into a requirements handoff.

Structure

Intent, 10 minutes. One person states the why in plain language, uninterrupted. The group's only job is to find where they disagree with it. No solution talk yet. The friction is scheduled here, not hoped for.

The walls, 15 to 20 minutes. Constraints, non-goals, blast radius. What must not change, which systems this touches, what the agent must not decide on its own. This is where system thinking transfers, because it's where the invisible constraints get named out loud.

The gaps, 20 to 30 minutes. The core, and where the behavior gets written. The group asks the questions the agent won't ask. Edge cases, failure modes, what happens when. Every answered question becomes a sentence of behavior in the spec. Every unanswerable one becomes an explicit open decision with an owner. By the end of this phase the what exists, built question by question instead of drafted upfront.

Read-back, 10 minutes. The driver reads the spec aloud, top to bottom. Then the lead asks each

person, by name: consent or objection. Silence isn't an answer, everyone answers. Objections get logged or resolved, and consent gets recorded.

The gate, 5 minutes. Intent stated. Non-goals listed. Edge cases answered or explicitly deferred. No sentence a new team member would misread. If the gate fails, the ticket goes back and the agent doesn't run.

Output

One spec document, versioned, in the repo next to the code. The sections mirror the session: Intent, Constraints and Non-Goals, Behavior and Edge Cases, Open Decisions. Open Decisions is the honest section, because it tells the agent and the reviewers where judgment was deferred on purpose rather than omitted by accident.

Between the gate and the run

The spec is visible before the agent runs, and the Stakeholder Navigator carries it to whoever the outcome touches. This is the review, moved. Stakeholders used to inspect the increment after the build, because the build was the expensive part, and now the spec is the increment worth inspecting. An objection now costs a conversation, an objection after the run costs the run, the review of the run, and a little trust. So the window is short, the default is the agent runs, and an objection is a new session, not a veto scribbled in a comment. Nobody approves the spec, because approval is a handoff wearing a safety vest.

The async adaptation

The template above assumes a room: synchronous, one screen. Distributed teams run the same gate as a review request on the spec, the form the Agora describes. The mapping: the draft carries intent, constraints, and non-goals before it opens. The comment thread is the challenge phase, and silence is still not an answer; every named reviewer responds or the spec doesn't merge. The consent round is the approval set, with objections as blocking comments. The timebox becomes a deadline: two working days, not two weeks. One failure mode travels with the form: a spec drafted by an agent and reviewed by an agent merges without anyone having understood it. The review request is the session; if no human thinks in it, it didn't happen. It is slower than a room and better than a document nobody owned. The fuller async build, as it evolves, lives at leftoftheloop.dev.

What the template doesn't solve

The run will surface things the room didn't. That's another session, not an edit, and it's why sessions have to stay cheap enough to re-run.

There is no estimation ritual anywhere, no points, no poker. The sizing question moved, it's no longer how long this will take, it's whether this is small enough to spec in one session. That sizing is the estimate, made together at the gate. Commitment moved with it. The team no longer commits to estimated capacity, it commits to specs through the gate, and that's what predictability measures now. The when comes from flow, how many gated specs the team finishes, counted instead of guessed. And the count only means something because the session cap keeps the units roughly comparable, the gate normalizes what it passes. The #NoEstimates crowd got here first, Woody Zuill again, with Vasco Duarte writing the book: make the items small and similar, and the counting does the forecasting.

A room can nod along the way a model does. The consent round forces everyone to answer, but consent can be performed just like silence can. The rotating lead helps, and it doesn't fix it.